NAME:Riya Dinesh Paturde

Batch: B2

Roll no: 24

Aim: Use sequence data types and their associated operations in Python

a) Strings b) Lists c) Arrays d) Tuples

e) Dictionary f) Sets g) Range

Theory:

1)What are sequence data types in Python? Name them and explain how indexing and slicing work across different sequence types?

Ans: A sequence data type is a collection of items stored in a specific order.

Each element has a position. Main sequence data types in python : List ,Tuple, String, Range ,

Array indexing in sequence:

Indexing means accessing element using position.

Indexing starts from 0.

Example: my _ list =[10,20,30,40] print(my _ list[0]) print(my _ list[2])

Slicing: slicing extracts a portion of a sequence

ex : sequence [start: stop: step]

2)What is the difference between a list and a tuple in python ? Why would you prefer a tuple over a list in real world application?

Ans: List is mutable ,tuple is immutable

Lists use [ ], tuple use( )

Tuples are preferred when data should not change , need faster execution , and less memory. use tuple because ,it is safer faster performance, used for fixed records

3)How are python arrays different from lists? When should you use the array module instead of a list?

Ans: List: can store different data type , More flexible Array: Store same data type only , Uses less memory, Faster for numerical data.

Use array when: Working with large numeric data . Memory efficiency is important.

4.Explain how dictionaries work internally in python. Why are dictionary lookup faster compared to lists or tuples?

Ans: Dictionary uses a hash table. Each key is converted into a hash value.

this hash determines where the value is stored . Example , d = {"name": "Riya", "age":18}

print(d["name"]) Riya Why it is faster: Direct access using hash. Time complexity is O(1) for lookup

5)What is the difference between sets and lists in python? How do sets handle duplicate values, and where are sets useful in practical scenarios?

Ans: List: An ordered collection that allows duplicate values. It uses [ ] ,it performance is slower Sets: An ordered collection that store only unique element. It uses { }, performance is faster.

```
#String Operation
print("\n String Operation ")
s = "Hello python"
print("original String:",s)
print("Upper Case:",s.upper())
print("Lower Case:",s.lower())
print("Length:",len(s))
print("substring:",s[0:5])

#List Operation
print("\n List Operation")
list1 = [10,20,30,40]
print("original list:",list1)
list1.append(50)
print("After append:",list1)
list1.remove(20)
print("After remove:",list1)
print("List slicing:",list1[1:3])

#Array Operation
print("\n Array Operation")
from array import*
arr = array('i',[1,2,3,4,5])
print("Array :",arr)
arr.append(6)
print("after append:",arr)
arr.remove(3)
print("after remove:",arr)

#Tuple Operation
print("\n Tuple Operation")
tup=(100,200,300,400)
print("tuple:",tup)
print("tuple length:",len(tup))
print("tuple slicing:",tup[1:3])
```

```
#Dictionary
print("\n Dictionary operation")
dict1={"name":"Riya","age":20,"City":"Amravti"}
print("Dictionary:",dict1)
dict1["age"]=20
print("after update:",dict1)
print("keys:",dict1.keys())
print("values:",dict1.values())

#sets
print("/n sets operation ")
set1={1,2,3,4,5}
set2={4,5,6,7,8}
print("set1:",set1)
print("set2:",set2)
print("union:",set1.union(set2))
print("intersection:",set1.intersection(set2))
print("difference:",set1.difference(set2))

#Range
print("/n Range operation")
range1=range(1,10,2)
print("range:",list(range1))
```

```
 String Operation
original String: Hello python
Upper Case: HELLO PYTHON
Lower Case: hello python
Length: 12
substring: Hello

 List Operation
original list: [10, 20, 30, 40]
After append: [10, 20, 30, 40, 50]
After remove: [10, 30, 40, 50]
List slicing: [30, 40]

 Array Operation
Array : array('i', [1, 2, 3, 4, 5])
after append: array('i', [1, 2, 3, 4, 5, 6])
after remove: array('i', [1, 2, 4, 5, 6])

 Tuple Operation
tuple: (100, 200, 300, 400)
tuple length: 4
tuple slicing: (200, 300)

 Dictionary operation
Dictionary: {'name': 'Riya', 'age': 20, 'City': 'Amravti'}
after update: {'name': 'Riya', 'age': 20, 'City': 'Amravti'}
keys: dict_keys(['name', 'age', 'City'])
values: dict_values(['Riya', 20, 'Amravti'])
/n sets operation
```

```
set1: {1, 2, 3, 4, 5}
set2: {4, 5, 6, 7, 8}
union: {1, 2, 3, 4, 5, 6, 7, 8}
intersection: {4, 5}
difference: {1, 2, 3}
/n Range operation
range: [1, 3, 5, 7, 9]
```

conclusion:

Sequence data types in python help in storing and organizing data in different forms.

by using these types together in programs, we can perform various operation like storing ,accessing , and processing data efficiently. These data types make python flexible and powerful for solving different types of operation.